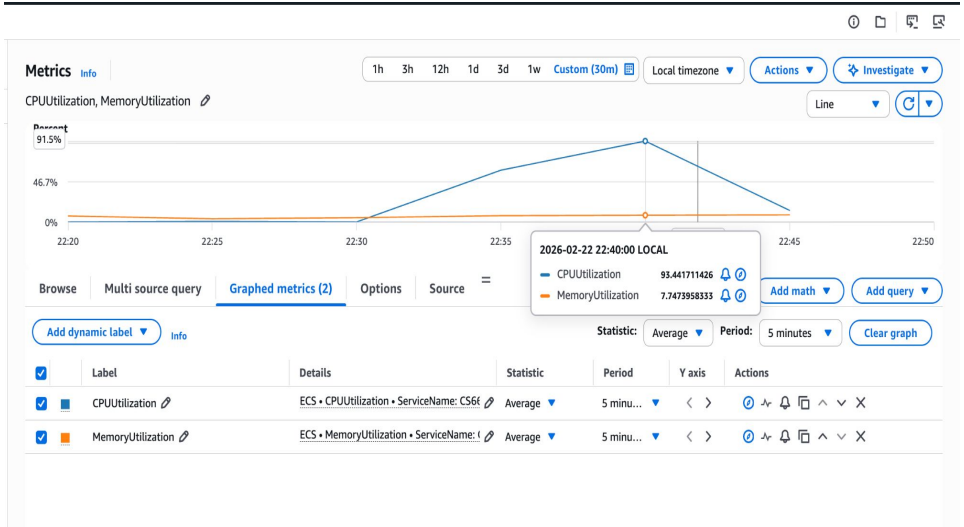
Midterm Mastery

Manasha Sekar

Part I - HW6 analysis

- 11,000 iterations of a complex math equation per product.
- Work that takes a fixed amount of CPU time regardless of how you write the code.
- Without it, 100 string comparisons take microseconds, and the CPU never stresses even with 200 users.
- With 5 users, CPU was ~59%, and the average response time was 289 ms.



With 20 users, CPU hit 93%, and response times degraded to 1,104 ms, a 4x increase. RPS barely changed (17.8 -18.2), showing CPU saturation.

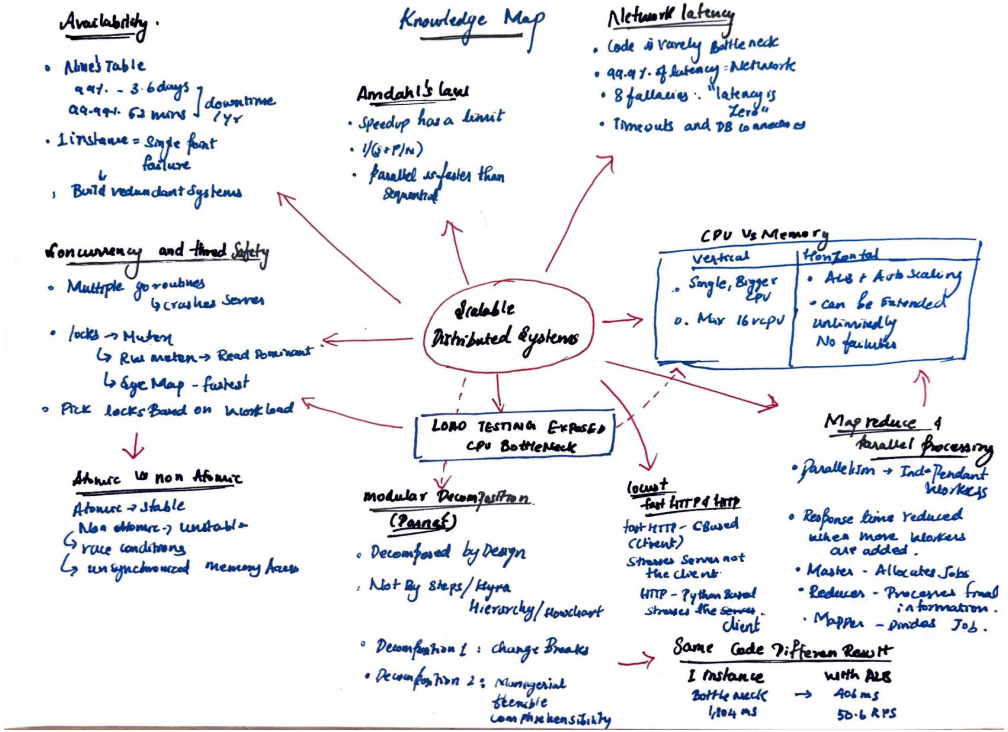
HW 6 - Auto Scaling

- Same 20 users that broke Part 2 now averaged 406 ms.
- Throughput jumped from 18.2 to 50.9 RPS.
- Auto-scaling kicked in when CPU exceeded 70%.
- Instances 2-3-4.
- Resilience testing,ALB rerouted traffic to healthy instances.
- 50 users- 0 failures- 4 instances.

	Part 2 (1 instance)	Part 3 - 20 users	Part 3 - 50 users
Avg Response	1,104ms	406ms	710ms
Median	1,200ms	290ms	290ms
RPS	18.2	50.9	65.3
Max	3,105ms	2,587ms	7,261ms
Failures	0	0	0

Part II - Key Concepts

- **Availability** — Nines table, single point of failure, redundant systems
- **Amdahl's Law** — Speedup has a limit, parallel vs sequential
- **Network Latency** — Code is rarely the bottleneck, 8 fallacies, timeouts
- **Concurrency and Thread Safety** — Goroutines, locks, mutex, sync.Map
- **Atomic vs Non Atomic** — Race conditions, unsynchronized memory
- **Modular Decomposition** — Decomposed by design, change breaks
- **Load Testing Exposed CPU Bottleneck** — Center of the map
- **CPU vs Memory** — Vertical vs Horizontal scaling
- **MapReduce & Parallel Processing** — Parallelism, master/reducer/mapper



Connection of homeworks

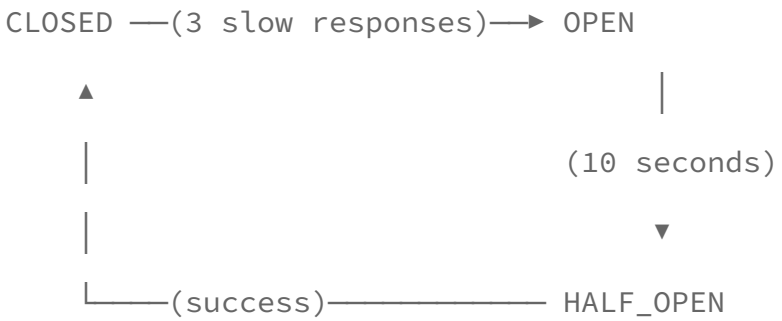
- HW3: sync.Map for concurrent writes → used in HW6 for product storage
- HW4: MapReduce showed parallelism — 3 mappers gave 3x speedup (0.84s → 0.28s)
- HW5: Load testing + containerization → Locust + Docker in HW6
- HW6: Everything together — concurrent Go service, containerized, deployed on Fargate, load tested, horizontally scaled

part III - The Problem: Dependency Degrades

- simulateWork() has a heavy mode (1,000 → 110,000 iterations)
- Simulates a slow dependency (degraded ML model, overloaded service)
- Need to detect the failure and skip the broken part.
- Scaling doesn't help here — every instance has the same broken work.

```
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -X POST http://CS6650L2-alb-1023593052.us-west-2.elb.amazonaws.com/toggle-heavy
{"work_mode":"heavy"}
(base) manashasekar@Manashas-MacBook-Air BSDS %
```

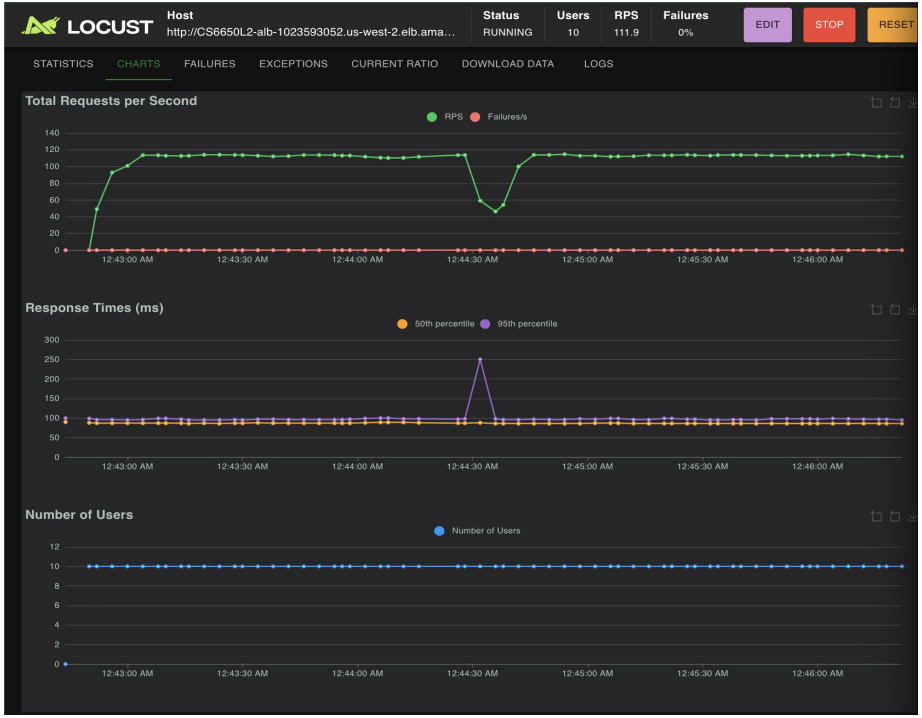
Circuit breaker



- CLOSED: Normal. Counts responses >300ms as failures
- OPEN: Skips simulateWork(), returns results instantly
- HALF_OPEN: After 10s, tests one request. Success → CLOSED

Results

Metric	Normal	Heavy (before trip)	After Circuit Opens
RPS	~115	~45	~115 (recovered)
Median	90ms	250ms+	90ms
Max	294ms	6,058ms	294ms
Failures	0%	0%	0%



Only 3 requests suffered before circuit opened

RECOVERY

```
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -X POST http://CS6650L2-alb-1023593052.us-west-2.elb.amazonaws.com/toggle-heavy
{"work_mode":"heavy"}
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -s "http://CS6650L2-alb-1023593052.us-west-2.elb.amazonaws.com/products/search?q=a
lpha" | grep -o '"circuit_state":"[^"]*","work_mode":"[^"]*"'
"circuit_state":"OPEN","work_mode":"heavy"
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -X POST http://CS6650L2-alb-1023593052.us-west-2.elb.amazonaws.com/toggle-heavy

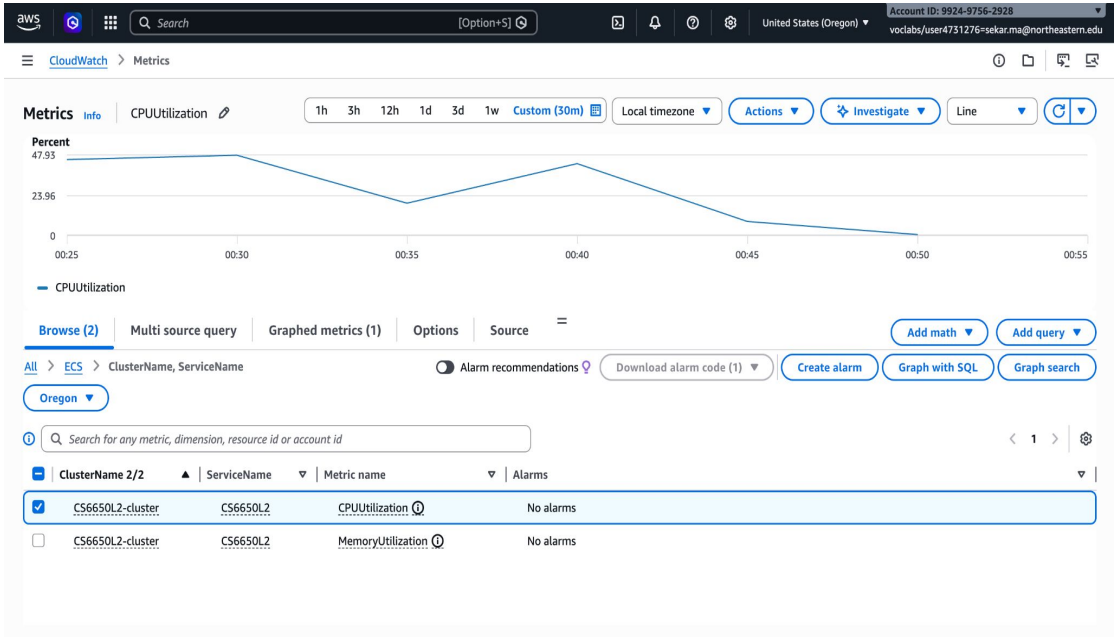
{"work_mode":"normal"}
(base) manashasekar@Manashas-MacBook-Air BSDS % sleep 10

(base) manashasekar@Manashas-MacBook-Air BSDS % curl -s "http://CS6650L2-alb-1023593052.us-west-2.elb.amazonaws.com/products/search?q=a
lpha" | grep -o '"circuit_state":"[^"]*","work_mode":"[^"]*"'

"circuit_state":"CLOSED","work_mode":"normal"
(base) manashasekar@Manashas-MacBook-Air BSDS %
```

OPEN, heavy → toggle normal → sleep 10 → CLOSED, normal

CPU UTILIZATION - CLOUDWATCH



CPU Utilization: ~48% during normal load, variation during heavy/circuit breaker cycle, then dropped after test stopped

Summary

- Scaling handles demand
- Circuit breaker handles failures
- CPU was the bottleneck: 18 → 65 RPS with horizontal scaling
- Circuit breaker: only 3 requests suffered, 0% failures, self-healed
- Every pattern has tradeoffs — choose based on your failure mode